In Game Messaging for a 3D Game Production Architecture and

Operating Model Date: 2025-08-10 UTC

In Game Messaging for a 3D Game Production Architecture and
Operating Model Date: 2025-08-10 UTC
One Pager Summary Goal Integrate fast reliable and safe in game messaging into a 3D title covering party squad team guild lobby and global channels with low latency delivery presence typing indicators moderation and persistence. The platform must scale across regions and titles without affecting the authoritative gameplay loop.

Key Outcomes

In region message delivery under one hundred milliseconds p95 with ordered per channel streams. Presence and typing updates under two hundred milliseconds p95. Reliable persistence of recent history with configurable retention and offline delivery. Strong moderation and abuse controls built into the hot path. Operational excellence with clear service level objectives canary rollouts and deep observability.

Primary Assumptions

Concurrent players in the hundreds of thousands to millions globally. Peak message rate tens of thousands to hundreds of thousands per second per busy region. Clients include PC console and mobile using secure websockets or QUIC over UDP. Authoritative gameplay servers are separate but integrated through events and notifications.

Functional Requirements

Channels Party squad team guild lobby match and global channels. Direct messages with privacy controls. Features Send receive edit and delete with audit. Presence typing read receipts optional. Rich text emojis stickers simple attachments within policy size limits. System announcements and server messages. Moderation Profanity and PII filters spam throttles word and user mutes shadow mutes kicks bans and report flows. Persistence Recent history per channel retention windows by scope. Offline delivery on reconnect. Export for compliance. Integration Game servers can post system messages and event hooks. UI widgets in engine for chat overlay and consoles. Configuration Per title and per environment settings including quotas rate limits and retention.
Non
Availability read and write APIs target ninety nine point nine five percent or better per region. Latency targets p95 message end to end in region under one hundred milliseconds and presence under two hundred milliseconds. Durability RPO zero for accepted messages and admin actions. RTO under fifteen minutes per region. Scalability linear by partitions and shards. Cost aware autoscaling. Data residency options by region. Compliance SOC two Type two GDPR and CCPA with privacy by default and subject rights.

Architecture Overview

Client and Engine Integration Game client integrates a messaging SDK that provides an in game overlay widgets and background connection management. The SDK exposes engine friendly APIs for Unity Unreal or custom engines and supports reconnect backoff telemetry capture and accessibility options. Edge and Gateway CDN and WAF terminate TLS. An API and Websocket Gateway authenticates tokens enforces per user and per title rate limits and applies basic schema validation. Sticky routing by user id and channel id reduces cross shard traffic. Messaging Core Regional Kafka or Pulsar topics with partitions keyed by tenant and channel id provide ordered durable logs. Producers are gateways. Consumers are fanout workers that update Redis hot inboxes and publish to per channel pub sub. A transactional outbox in the gateway provides idempotency. Presence Service Presence state is kept in Redis with short TTL heartbeats and per room sets. Gateways emit join and leave signals during connect and disconnect and a guard prevents ghost sessions. Moderation Pipeline Inline fast filters for profanity lists and spam bursts. Asynchronous deeper checks for PII and ML models with quarantine

queues and moderator review. Policies are applied before fanout to avoid wide distribution of abusive content. Persistence DynamoDB Scylla or Cassandra holds durable notifications and read state per user

and channel with partition keys tenant id user id and sort keys by time. Redis keeps a capped history window for hot reads. Object storage holds cold archives and exports. Integration with Gameplay Gameplay servers post system messages and match events through a signed service channel. Messaging never blocks the authoritative tick. Clients may optionally surface lightweight alerts in the HUD with a texture atlas friendly layout.
Data Flow Producer Client sends message to Gateway which authenticates validates and writes to outbox. Outbox writer publishes to Kafka partition keyed by tenant and channel id. Consumer Fanout worker consumes validates moderation rules updates Redis history and counters persists to the durable store in batches and publishes to per channel pub sub. Websocket Gateway pushes to all online subscribers. Reconnect On reconnect the SDK downloads missed messages using a since token and resynchronizes presence and counters.
Technology Stack reference Edge and auth CDN WAF API Gateway OAuth two point one or OpenID Connect and mTLS for service to service. Streaming Kafka with Schema Registry or Pulsar. Kafka Connect for sinks to storage and lake. Processing Go or Rust services with gRPC. Flink optional for analytics taps and rate calculation. Hot store Redis Cluster for lists sorted sets and counters. Durable store DynamoDB or Scylla or Cassandra for messages and state. Postgres or Aurora for admin metadata. Search OpenSearch for moderator search and case tooling. Analytics S3 or GCS with Parquet and Iceberg or Delta. ClickHouse or BigQuery for dashboards. Platform Kubernetes with autoscaling service mesh with mutual TLS. Argo CD and Argo Rollouts. Observability OpenTelemetry Prometheus Grafana Loki Jaeger. SIEM for security log correlation.

Ingestion

lag processor throughput Redis ops
p95 and p99 messaging latency websocket fanout time push success rate token health DLQ depth. Logs and traces Structured logs with tenant id user id hash request id and correlation id. Trace messaging across gateway fanout and client receive. SLO aware autoscaling Scale on latency and stream lag. Alerts on error budget burn skewed partitions hot keys and cold caches. Runbooks Cache cold start shard failover partition rebalancing regional read only mode and edge bootstrap.
Capacity Planning and Cost Estimate Reference region with ten million daily active users and fifty thousand messages per second sustained peaking at two hundred thousand. Kafka six to nine brokers one hundred to three hundred partitions replication three. Redis twelve to twenty four shards under sixty five percent memory. NoSQL store provisioned for write heavy workloads with adaptive capacity. Rough monthly cost per busy region from thirty thousand to eighty thousand depending on retention and analytics scans.

Storage Strategy

Hot path Redis lists for recent channel history capped per scope. Sorted sets for last
update per channel and per user unread counters. Hashes for payloads and compact metadata. Short TTLs to control memory growth. Durable path NoSQL store for message records with per user partitions for inbox patterns and per channel partitions for history. Conditional writes prevent duplicates. Batch writers reduce write amplification. Search path OpenSearch index for active moderator cases. Cold windows are queryable via analytics engines. Cold storage Object storage in Parquet partitioned by day and channel with compaction jobs.
Scaling Approach Partition topics by tenant and channel hash to preserve per channel order and maximize parallelism. Fanout workers scale with partitions. Redis shards scale horizontally with consistent hashing and hash tags to collocate related keys. Websocket gateways autoscale by active connections and p95 latency. Large channels use tiered fanout trees or channel partitions to avoid hot keys.

Data Models

Message tenant id channel id message id user id sequence created at edited at deleted flag
type text or system payload json moderation flags parent id for replies attributes. Channel tenant id

channel id type party team guild lobby global members retention policy slow mode created at. Presence tenant id user id status last heartbeat at session id node id. UserModeration tenant id user id mutes bans scopes expires at reason. DeliveryReceipt message id user id delivered at read at optional. AuditEvent actor tenant id action target attributes timestamp.

Latency Budgets and SLAs

Edge and TLS termination under twenty milliseconds p95. Gateway enqueue under fifteen milliseconds p95. Fanout processing under twenty milliseconds p95. Redis read or write under three milliseconds p95 same AZ. Websocket push under twenty milliseconds p95. End to end delivery under one hundred milliseconds p95 in region. Availability ninety nine point nine five percent for messaging APIs and websocket connections.

Key Algorithms and Workflows

Idempotency and ordering Event id and sequence per channel. Conditional operations in Redis and durable store to avoid duplicates. Partitioning ensures per channel ordering. Around me and paging Fetch rank then slice neighbors using list ranges with small hydration lookups.

Slow mode and rate limits Token buckets per user and per channel with exemptions for system roles. Graceful backoff errors. Typing indicators and presence Lightweight signals through pub sub with short TTLs and collapse on burst. Attachments Upload via pre signed URLs to object storage with size and type checks. Message references the object id. Thumbnails generated asynchronously. Moderation and safety Inline blacklist spam detection shadow mute fast paths. Deeper ML checks in async pipeline. Appeals and audit trails in admin console.

Security Reliability and Compliance Security TLS one point three in transit and KMS at rest. Pseudonymous ids by default. OAuth scopes RBAC mTLS inside mesh signed webhooks for gameplay integration strict CORS for public APIs. DDoS aware relays and rate limits. Privacy and compliance Data retention by scope export and deletion APIs and optional data residency controls. Audit logs and consent in account system where applicable. Reliability Multi AZ for every tier with replay and backpressure via Kafka. DLQs and quarantine queues. Disaster recovery with tested backups and object storage versioning. Zero blocking on gameplay loop. Change management Canary releases feature flags progressive traffic shifting automatic rollback on SLO burn.

Multi Tenancy Tenant id in every topic key and row. Quotas and budgets per tenant. Soft isolation by partitions namespaces and rate limits. Optional hard isolation with dedicated cells VPCs and keys for large tenants.

CI CD and Infrastructure as Code Terraform or Pulumi and Helm for clusters topics shards and policies. Argo Rollouts for canary and rollback based on SLOs and error budgets. Schema evolution via registry and contract tests. Online DDL for relational metadata with dual read windows.

APIs and Integration Points Client POST message GET history GET counters POST typing POST read receipt Websocket subscribe to channel and presence. Server and tools POST system message POST moderator action GET search GET audit events. Integrations Push providers for mobile optional Slack or webhook for enterprise tenants analytics export to lake.

Observability and Operational Practices Golden dashboards

Testing Strategy and Failure Drills

Unit and property tests for ordering dedupe and counters. Contract tests for schemas and public APIs. Load tests for hot channels and bursty writes. Soak tests to detect memory fragmentation. Chaos drills for broker loss Redis shard failover websocket gateway churn and regional failover. Game days with cross shard migration and partial outages.

Phased Build Plan

MVP three months Single region single tenant websockets per channel party and match chat Redis hot history DynamoDB durable store profanity lists basic moderation minimal search and admin tools. Phase two six to nine months Multi region cells guild and global channels presence and typing canary delivery advanced moderation and abuse workflows analytics taps search and improved

retention. Phase three twelve months Hard isolation for premium tenants ML assisted moderation tiered fanout for very large channels regional data residency and stricter SLOs.
Key Choices Trade Offs Risks and Mitigations Redis lists and sorted sets deliver required latency but need careful key design and memory hygiene. Keep payloads small and keys short and cap lists. Exactly once semantics across services are impractical. Use outbox idempotency keys and conditional writes for effective once behavior. Hybrid fanout balances cost and latency. Pure fanout on write can explode storage while pure fanout on read can increase latency for online users. Hot channels and celebrity users create skew. Mitigate with tiered fanout batching and careful key hashing. Push providers and networks have variability. Use retries circuit breakers and multi provider failover where needed.